

1. Plan SQA preliminar según IEEE 730

1.1. §1 Purpose (Propósito y alcance)

El presente Plan de Aseguramiento de la Calidad del Software (SQA) se elabora conforme al estándar IEEE 730-2014 [1] para el proyecto “**Sistema de Gestión de Proyectos de Investigación (SGPI)**”.

Propósito: Establecer las actividades, responsabilidades, métricas y procedimientos que garantizarán que el software desarrollado cumpla con los requisitos funcionales y no funcionales definidos, especialmente aquellos priorizados según ISO/IEC 25010. El plan busca asegurar la calidad durante todo el ciclo de vida del proyecto, desde la especificación hasta la entrega final.

Alcance: Aplica a todos los entregables del proyecto SGPI: código fuente, documentación técnica, manuales de usuario, informes de pruebas, y cualquier artefacto generado por el equipo de desarrollo. Cubre las fases de análisis, diseño, implementación, pruebas y despliegue. No cubre el mantenimiento posterior a la entrega final, aunque se recomienda extender el plan para futuras fases.

1.2. §3 Management (Roles y responsabilidades)

Los roles definidos para la ejecución del plan SQA se muestran en la Tabla 1.

Cuadro 1: Roles y responsabilidades en el plan SQA

Rol	Responsabilidades
Gerente de Proyecto	Aprobar el plan SQA, asignar recursos, supervisar el cumplimiento de hitos de calidad.
Líder de Calidad (SQA)	Ejecutar auditorías internas, medir KPIs, reportar no conformidades, gestionar el repositorio de métricas.
Desarrolladores	Seguir estándares de codificación, realizar pruebas unitarias, documentar su trabajo.
Analista de Pruebas	Diseñar y ejecutar casos de prueba, reportar incidentes, mantener la trazabilidad.
Cliente / Usuario clave	Validar criterios de aceptación, reportar problemas en fase de pruebas de usuario.

1.3. §5 Standards, practices, metrics

1.3.1. Estándares aplicados

- **Codificación:** Guía de estilo basada en PEP 8 (Python) y ESLint (JavaScript).
- **Documentación:** Formato uniforme en $\text{\LaTeX}$, con plantilla institucional.
- **Pruebas:** Estructura basada en ISTQB v4.0 [2] para casos de prueba.
- **Versiónamiento:** Git con flujo *Git Flow* (rama develop, feature, release, main).

1.3.2. Prácticas obligatorias

- Todo *commit* debe incluir un identificador de tarea (JIRA o GitHub Issues).
- Las *pull requests* requieren revisión por al menos un compañero y superar análisis estático (SonarQube).
- Las reuniones semanales incluirán una revisión de los KPIs de calidad.

1.3.3. Métricas (KPIs)

Los KPIs definidos en la sección 2.3 (ISO/IEC 25023) se medirán con las siguientes frecuencias:

- **Disponibilidad:** diaria (Grafana/Prometheus).
- **Integridad de datos:** por transacción crítica (logs estructurados).
- **Cobertura de pruebas:** al final de cada *sprint* (pytest-cov / JaCoCo).
- **Tiempo de respuesta:** semanal (Apache JMeter).
- **Usabilidad (tasa de éxito):** después de cada prueba con usuarios (medición manual).
- **Mantenibilidad (deuda técnica):** por cada *commit* (SonarQube).

1.4. §7 Test (Estrategia de pruebas)

La estrategia de pruebas sigue un modelo basado en niveles:

1. **Pruebas unitarias:** Realizadas por los desarrolladores sobre cada función/módulo. Cobertura mínima exigida: 80 % de líneas de código. Herramienta: JUnit/Pytest.
2. **Pruebas de integración:** Validan la interacción entre módulos (API, base de datos). Se ejecutan en cada integración continua. Herramienta: Postman/Newman.
3. **Pruebas de sistema:** End-to-end sobre requisitos funcionales y no funcionales (rendimiento, seguridad básica). Herramientas: Selenium, JMeter.
4. **Pruebas de aceptación:** Definidas junto al cliente mediante criterios de aceptación de las historias de usuario. Se realizan en un entorno de preproducción.

Automatización: Se utiliza GitHub Actions para ejecutar automáticamente las pruebas unitarias y de integración en cada *push* a las ramas **develop** y **main**. Los informes de pruebas se almacenan como artefactos.

1.5. §8 Problem reporting (SLA por severidad)

Todo problema (defecto, falla, incumplimiento) debe reportarse mediante el sistema de seguimiento (GitHub Issues). Se asignan los siguientes niveles de severidad y SLA:

El proceso de reporte incluye: apertura del *issue* con etiqueta de severidad, asignación a un desarrollador, actualización diaria del estado y notificación al Líder de Calidad. Los problemas críticos deben escalar inmediatamente al Gerente de Proyecto.

Cuadro 2: SLA para reporte de problemas según severidad

Severidad	Descripción	Tiempo respuesta inicial
Crítica (S1)	Impide totalmente el uso del sistema; pérdida de datos.	2 horas
Alta (S2)	Funcionalidad principal afectada; no hay <i>workaround</i> .	8 horas
Media (S3)	Funcionalidad secundaria afectada; existe <i>workaround</i> .	2 días hábiles
Baja (S4)	Error estético, texto incorrecto, mejora menor.	5 días hábiles

Referencias

- [1] IEEE Computer Society. (2014). *IEEE Standard for Software Quality Assurance Processes (IEEE Std 730-2014)*. IEEE. DOI: 10.1109/IEEESTD.2014.6835311.
- [2] International Software Testing Qualifications Board. (2023). *ISTQB Foundation Level Syllabus v4.0*. ISTQB.
- [3] Galin, D. (2018). *Software Quality: Concepts and Practice*. Wiley-IEEE Computer Society Press. (Capítulos 21–22).
- [4] Pressman, R. & Maxim, B. (2020). *Software Engineering: A Practitioner’s Approach* (9^a ed.). McGraw-Hill. (Capítulo 24 sobre SQA).